

Scalable Concert Ticket Acquisition System

Informe final: arquitectura directa REST e indirecta RabbitMQ

Entorno de pruebas: VM AWS Academy como cliente, torre como servidor, comunicación por Tailscale.

Backend compartido: Redis. Cola de mensajería indirecta: RabbitMQ.

Nombre: Jordà Coca Rodríguez

Profesor: Usama Benabdelkrim Zakan

1. Resumen ejecutivo

El proyecto implementa y evalúa un sistema escalable de adquisición de entradas para conciertos. Se comparan dos estrategias de comunicación: una arquitectura directa basada en peticiones HTTP síncronas mediante FastAPI, y una arquitectura indirecta basada en RabbitMQ, donde el cliente publica solicitudes y un conjunto de workers las procesa de forma asíncrona.

Ambas arquitecturas usan Redis como estado compartido para el inventario, la asignación de entradas y las métricas. En las pruebas de correctitud no se observa overselling ni ventas duplicadas. La arquitectura indirecta presenta una mejora muy clara de throughput, especialmente al aumentar el número de workers, ya que RabbitMQ desacopla la entrada de peticiones del procesamiento.

La arquitectura directa se comporta correctamente, pero queda limitada por el camino HTTP cliente -> load balancer -> worker -> Redis. En la variante directa optimizada el throughput queda prácticamente plano alrededor de 123-124 req/s. En cambio, la arquitectura indirecta alcanza 715.85 msg/s con 1 worker y 4368.78 msg/s con 16 workers en la prueba de escalado.

2. Arquitectura implementada

2.1 Arquitectura directa REST

En la arquitectura directa, la VM cliente envía peticiones HTTP al load balancer FastAPI desplegado en la torre/local. El load balancer distribuye las solicitudes entre workers REST. Cada worker accede a Redis para consultar y modificar el estado del sistema. El cliente queda bloqueado hasta recibir la respuesta, por lo que el tiempo de respuesta incluye todo el camino de red, balanceo, procesamiento y acceso a Redis.

Flujo simplificado: Cliente VM -> Tailscale -> Load Balancer REST -> Worker REST -> Redis -> Respuesta HTTP.

2.2 Arquitectura indirecta RabbitMQ

En la arquitectura indirecta, la VM cliente no llama directamente a los workers. Publica mensajes JSON en RabbitMQ, dentro de la cola ticket_queue. Los workers MQ consumen mensajes de esa cola y actualizan Redis. Esta aproximación desacopla la velocidad de entrada de solicitudes de la velocidad de procesamiento.

Flujo simplificado: Cliente VM -> RabbitMQ -> Workers MQ -> Redis.

El load balancer de la parte indirecta se mantiene para operaciones de control y observabilidad, como resetear el estado, escalar manualmente workers en los tests sin autoscaler y consultar métricas. No forma parte del camino principal de procesamiento de tickets, ya que las compras se publican en RabbitMQ.

3. Validación de correctitud

La correctitud se evalúa con dos propiedades fundamentales: que no se vendan más entradas no numeradas que las disponibles y que no existan ventas duplicadas de un mismo asiento numerado.

Tabla 1. Correctitud en arquitectura directa

test	requests	success	fail	http_error	duplicate_success_seats	valid
unnumbered_no_overselling	25000	20000	5000	0	0	True
numbered_mod_1000_req_100_seats	1000	100	900	0	0	True

Tabla 2. Correctitud en arquitectura indirecta

test	requests	success	fail	processed	duplicate_success_seats	valid
unnumbered_no_overselling	25000	20000	5000	25000	0	True
numbered_mod_1000_req_100_seats	1000	100	900	1000	0	True

En la prueba unnumbered se envían 25.000 solicitudes sobre un inventario de 20.000 entradas. El resultado esperado es 20.000 ventas correctas y 5.000 rechazos. Ambas arquitecturas cumplen esta condición. En la prueba numbered se envían 1.000 solicitudes sobre 100 asientos distintos repetidos. El resultado correcto es 100 ventas y 900 rechazos. También se cumple en ambos casos, con duplicate_success_seats igual a 0.

4. Resultados de la arquitectura directa

4.1 Escalabilidad sin autoscaler

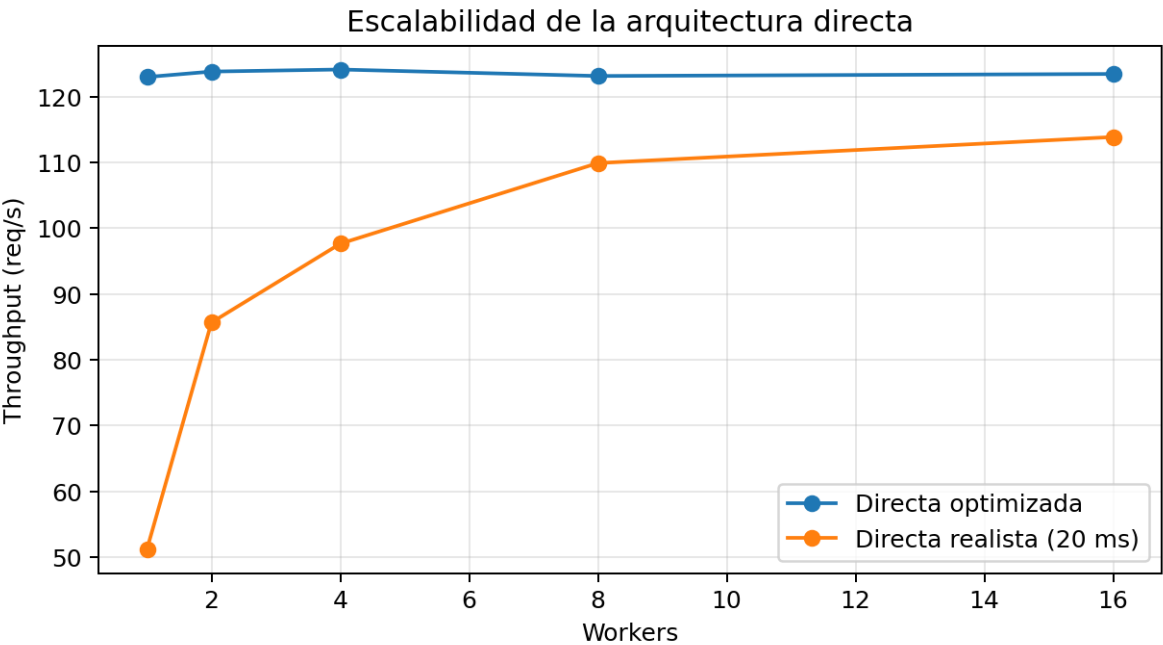


Figura 1. Escalabilidad de la arquitectura directa en modo optimizado y realista.

Tabla 3. Throughput directo por número de workers

mode	workers	throughput_req_s	latency_avg_ms	valid
optimized	1	123.002	258.778	True
optimized	2	123.831	257.473	True
optimized	4	124.145	256.492	True
optimized	8	123.158	258.936	True
optimized	16	123.467	258.208	True
realistic_20ms	1	51.245	622.996	True
realistic_20ms	2	85.682	372.409	True
realistic_20ms	4	97.664	326.971	True
realistic_20ms	8	109.918	290.428	True
realistic_20ms	16	113.882	280.506	True

En modo optimizado el throughput se mantiene prácticamente constante, alrededor de 123-124 req/s, aunque se aumente el número de workers. Esto indica que el cuello de botella no está en la cantidad de workers, sino en otro punto común del camino: la VM cliente, el canal Tailscale, el load balancer, Redis o el coste de cada petición HTTP.

En modo realista, donde se introduce una latencia artificial de 20 ms en el procesamiento, sí se aprecia escalado inicial: de 51.25 req/s con 1 worker a 113.88 req/s con 16 workers. A partir de 8 workers la mejora se reduce, lo que sugiere que el sistema se aproxima a un techo global.

4.2 Contención directa

Tabla 4. Contención en arquitectura directa

test	requests	throughput_req_s	success	fail	duplicate_success_seats	valid
contention_normal_20000	20000	122.336	20000	0	0	True
contention_hotspot_80_5	20000	123.181	4599	15401	0	True
contention_limited_100	20000	124.311	100	19900	0	True
contention_limited_20	20000	123.03	20	19980	0	True
contention_single_seat	20000	123.385	1	19999	0	True

Los escenarios de contención concentran muchas solicitudes sobre pocos asientos. En todos los casos se mantiene duplicate_success_seats igual a 0. El número de éxitos queda limitado por el número de asientos realmente disponibles en cada escenario: 100, 20 o 1 en los casos limited_100, limited_20 y single_seat.

5. Resultados de la arquitectura indirecta

5.1 Escalabilidad sin autoscaler

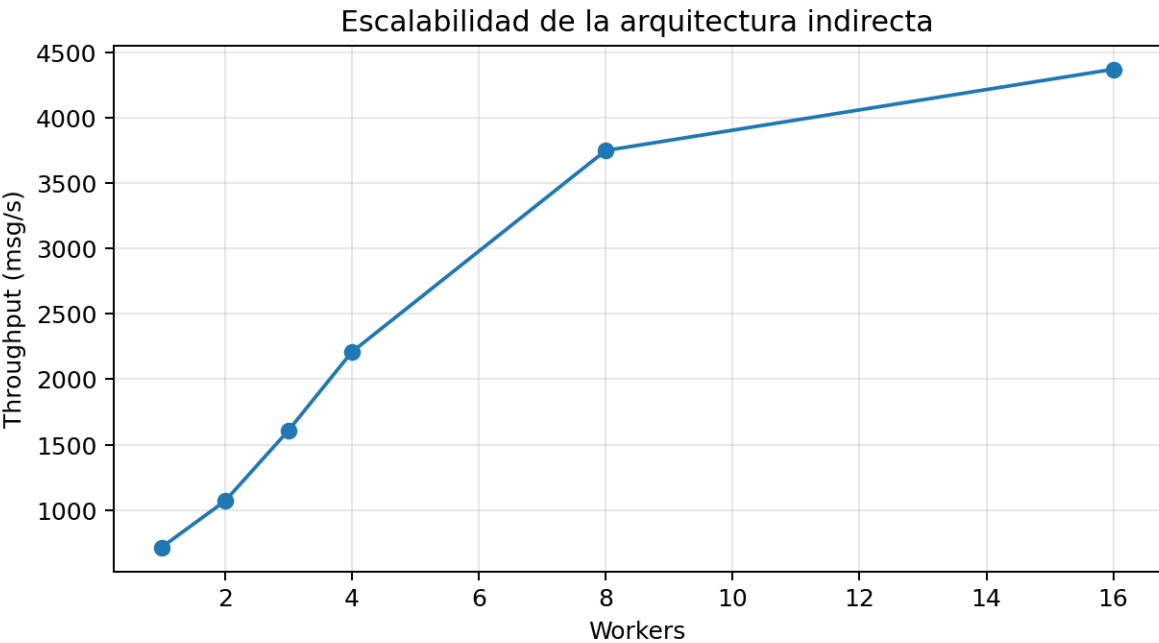


Figura 2. Escalabilidad de la arquitectura indirecta RabbitMQ sin autoscaler.

Tabla 5. Throughput indirecto por número de workers

workers	requests	elapsed_s	throughput_msg_s	success	fail	processed	duplicate_success_seats
1.0	50000.0	69.847	715.851	20000.0	30000.0	50000.0	0.0
2.0	50000.0	46.771	1069.041	20000.0	30000.0	50000.0	0.0
3.0	50000.0	31.087	1608.39	20000.0	30000.0	50000.0	0.0
4.0	50000.0	22.64	2208.484	20000.0	30000.0	50000.0	0.0
8.0	50000.0	13.34	3748.117	20000.0	30000.0	50000.0	0.0
16.0	50000.0	11.445	4368.783	20000.0	30000.0	50000.0	0.0

En esta prueba se publican 50.000 mensajes unnumbered para cada número de workers. Como el inventario tiene 20.000 entradas, el resultado correcto no es 50.000 éxitos, sino 20.000 éxitos y 30.000 rechazos. El script marcó estas filas como FAIL porque la expectativa inicial era incorrecta, pero la interpretación funcional es correcta: no hay overselling y processed=50.000.

La mejora de throughput es muy marcada: 715.85 msg/s con 1 worker, 1069.04 msg/s con 2 workers, 2208.48 msg/s con 4 workers, 3748.12 msg/s con 8 workers y 4368.78 msg/s con 16 workers. Esto demuestra que la arquitectura indirecta aprovecha mejor el paralelismo: RabbitMQ absorbe la ráfaga de entrada y reparte mensajes entre consumidores.

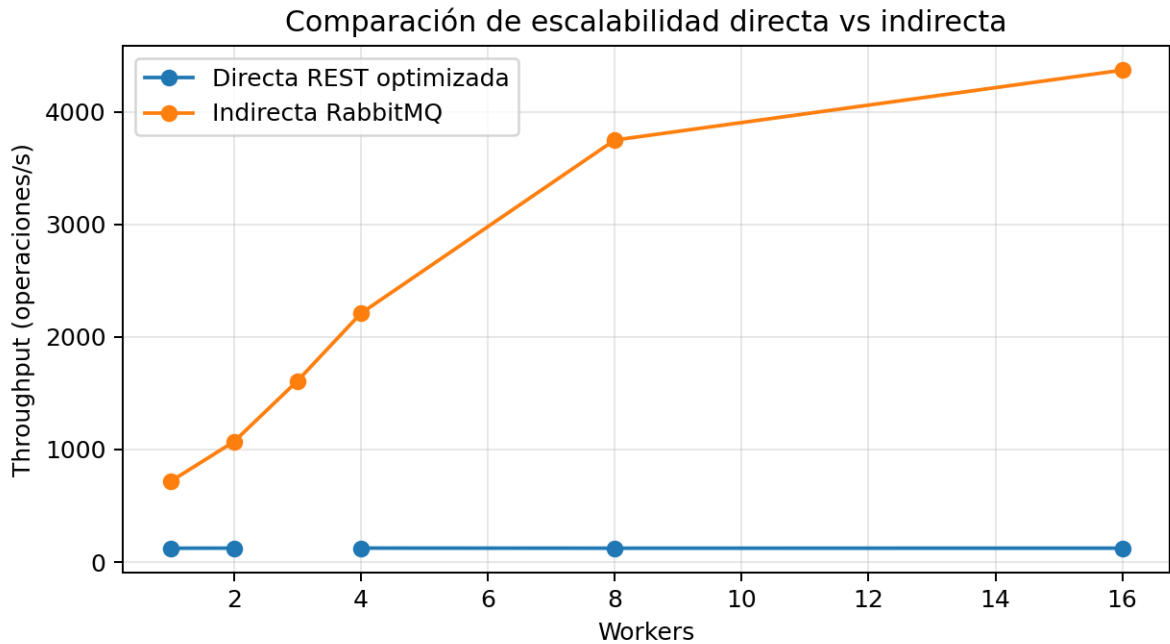


Figura 3. Comparación de throughput: directa REST optimizada frente a indirecta RabbitMQ.

La comparación muestra una diferencia importante. La directa optimizada queda estable alrededor de 123 req/s, mientras que la indirecta aumenta con el número de workers hasta superar 4.000 msg/s. La explicación principal es el desacoplamiento: en RabbitMQ el productor puede publicar rápido y los consumidores procesan en paralelo desde la cola, mientras que en REST cada petición arrastra el coste síncrono de la respuesta.

5.2 Contención indirecta

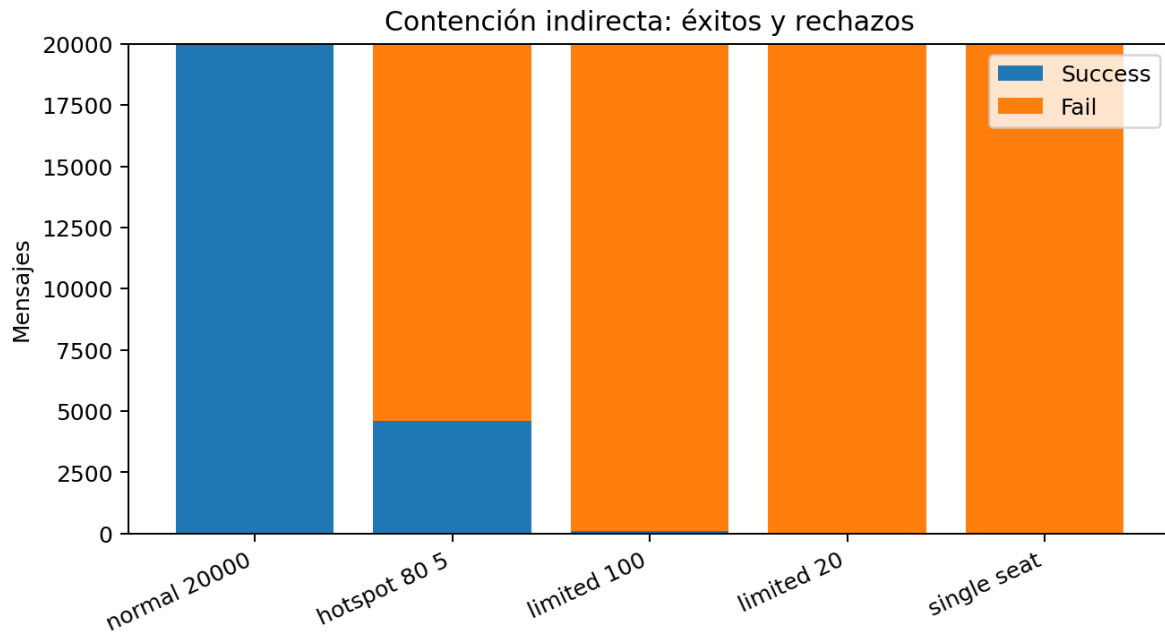


Figura 4. Contención indirecta: distribución de éxitos y rechazos.

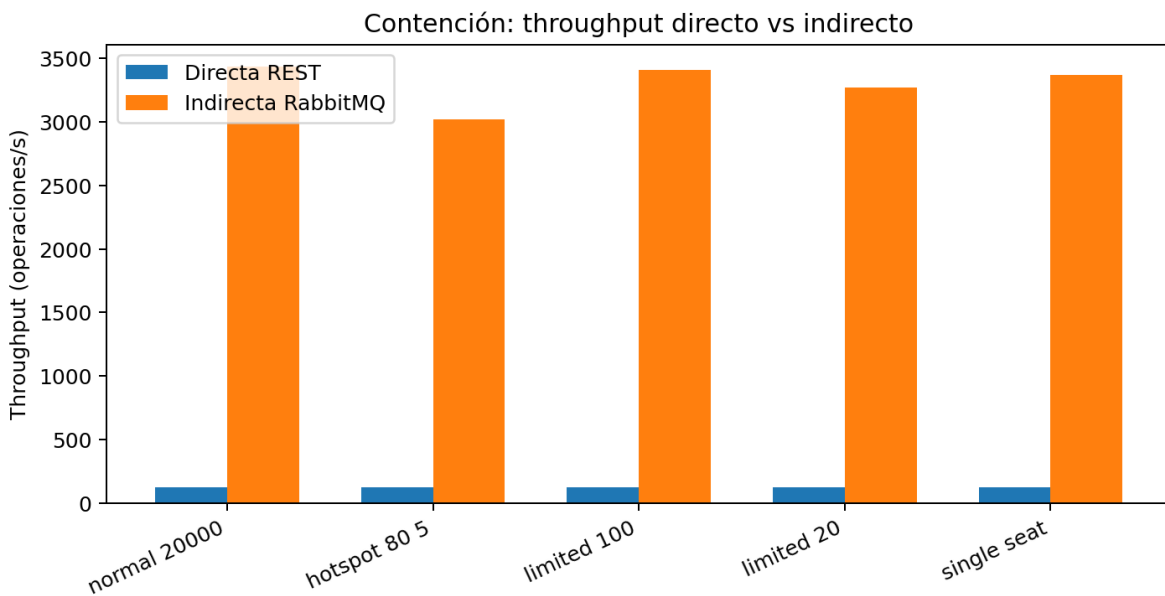


Figura 5. Comparación de throughput bajo contención: directa REST vs indirecta RabbitMQ.

Tabla 6. Contención en arquitectura indirecta

test	request s	throughput_msg_ s	succes s	fail	unique_sold_seat s	duplicate_success_seat s	valid
contention_normal_20000	20000	3435.843	20000	0	20000	0	True
contention_hotspot_80_5	20000	3017.537	4599	15401	4599	0	True
contention_limited_100	20000	3407.785	100	19900	100	0	True
contention_limited_20	20000	3270.074	20	19980	20	0	True
contention_single_seat	20000	3369.308	1	19999	1	0	True

				9		
--	--	--	--	---	--	--

La contención indirecta confirma la consistencia del sistema. En normal_20000 se venden las 20.000 entradas sin fallos. En hotspot_80_5 solo se consiguen 4.599 ventas porque muchas solicitudes compiten por los mismos asientos calientes; el resto se rechaza correctamente. En limited_100, limited_20 y single_seat los éxitos coinciden con el número de asientos únicos disponibles.

El throughput de hotspot_80_5 baja respecto a normal_20000, de 3435.84 msg/s a 3017.54 msg/s. La caída existe, pero no es extrema porque el worker normal usa SETNX/fail-fast: cuando un asiento ya está vendido, el rechazo se resuelve rápidamente.

6. Autoscalers

6.1 Autoscaler directo

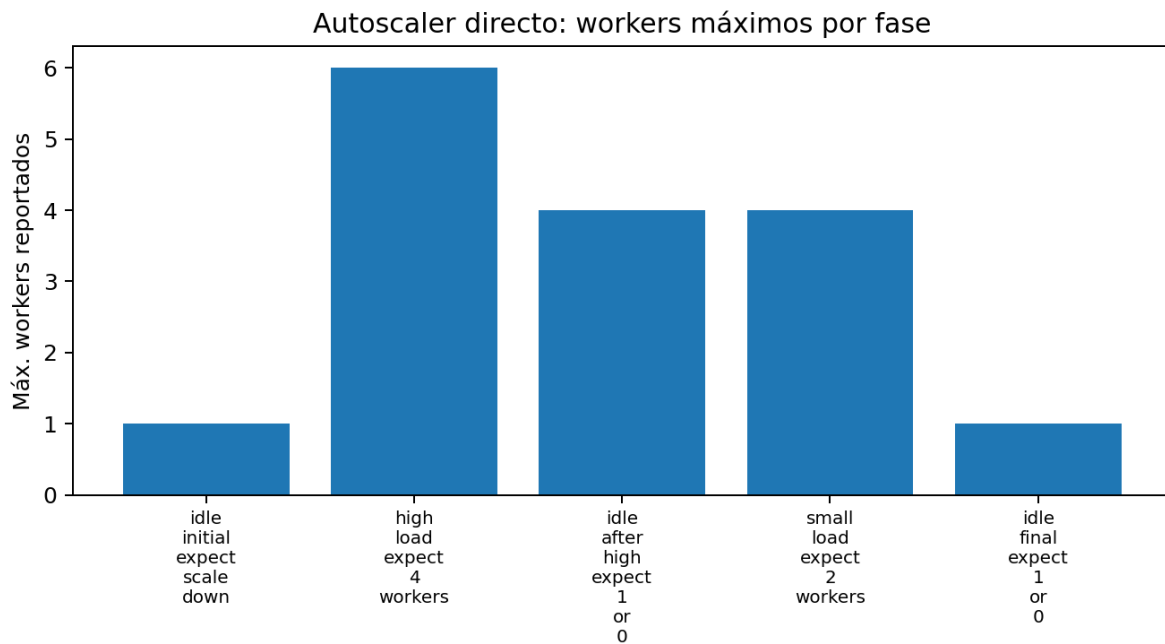


Figura 6. Autoscaler directo: máximo de workers reportados por fase.

Tabla 7. Resumen del autoscaler directo

phase	target_rps	expected_workers	max_reported_workers	max_pending	phase_throughput_processed_rps	lat_avg_ms
idle_initial_expect_scale_down	0.0	1	1	0	0.0	0.0
high_load_expect_4_workers	200.0	4	6	67	126.728	1900.597
idle_after_high_expect_1_or_0	0.0	1	4	0	0.0	0.0
small_load_expect_2_workers	90.0	2	4	4	83.384	480.358
idle_final_expect_1_or_0	0.0	1	1	0	0.0	0.0

El autoscaler directo responde a cargas altas aumentando el número de workers. En la fase high_load_expect_4_workers se esperaba llegar a 4 workers, pero el máximo observado fue 6. Esto se puede interpretar como sobreescalado moderado causado por la latencia acumulada, el backlog pendiente y la

variabilidad del sistema. Cuando la carga desaparece, el sistema reduce workers progresivamente hasta volver al estado mínimo.

6.2 Autoscaler indirecto

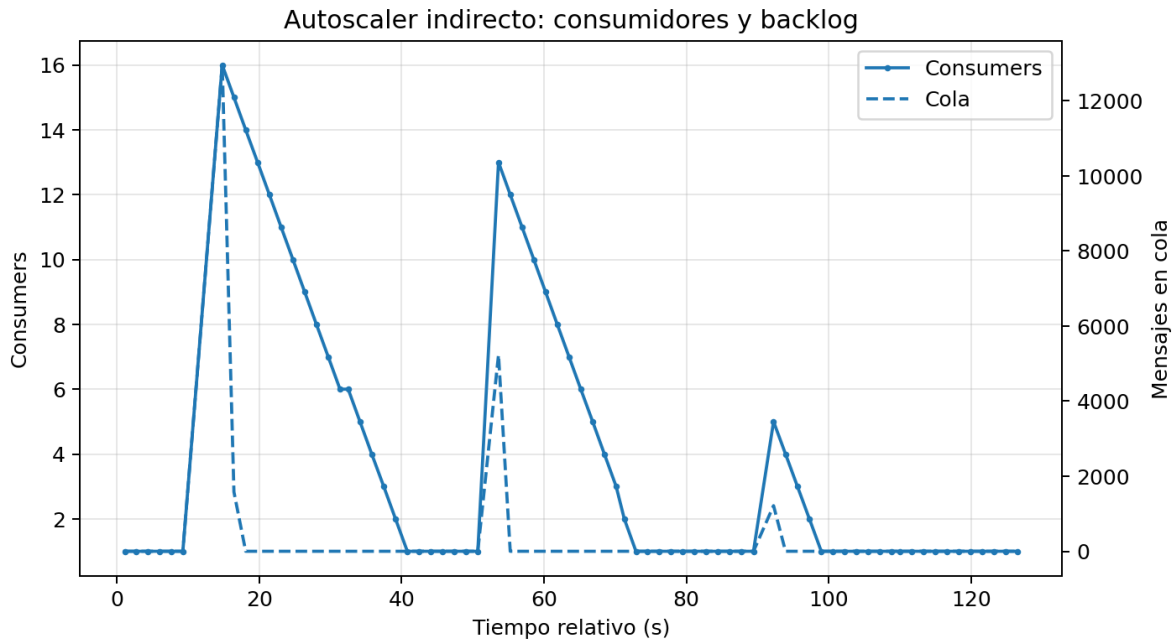


Figura 7. Autoscaler indirecto: consumidores y mensajes en cola a lo largo del tiempo.

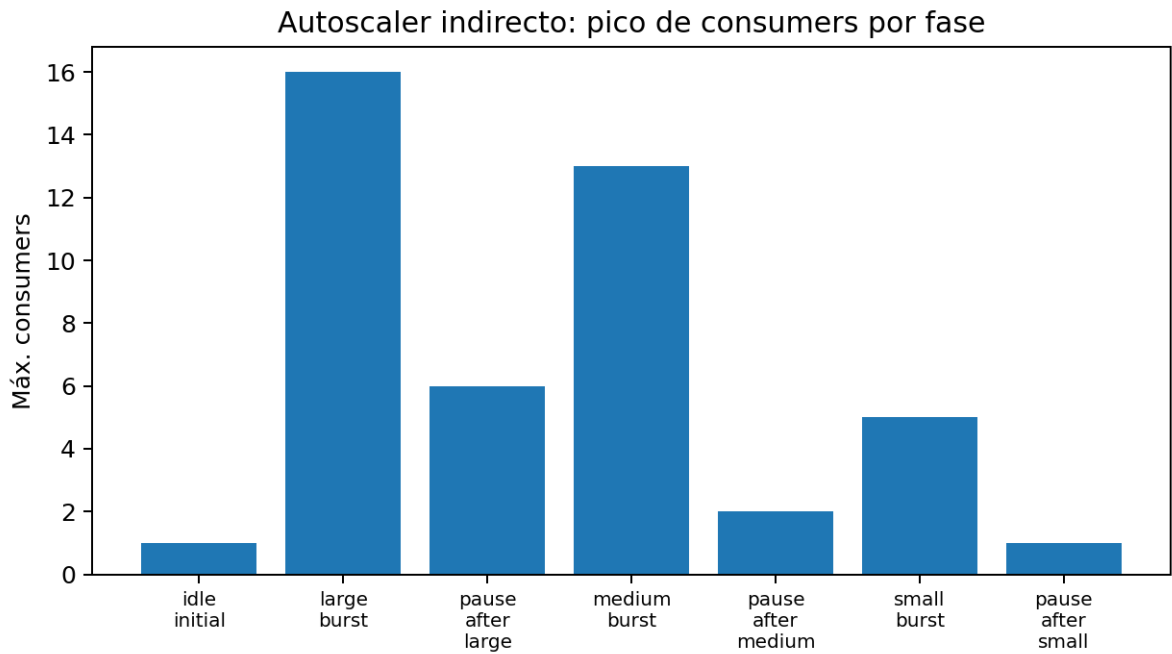


Figura 8. Autoscaler indirecto: pico de consumers por fase.

Tabla 8. Resumen del autoscaler indirecto

phase	workload	max_queue	max_expected_workers	max_actual_consumers	final_queue	final_consumers
idle_initial	0	0	1	1	0	1
large_burst	30000	12949	16	16	0	7
pause_after_large	0	0	1	6	0	1
medium_burst	8000	5228	11	13	0	4
pause_after_medium	0	0	1	2	0	1
small_burst	3000	1214	3	5	0	1
pause_after_small	0	0	1	1	0	1

El autoscaler indirecto ajusta el número de workers en función del backlog de RabbitMQ. La regla usada es $\text{ceil}(\text{queue_depth} / 500)$, limitada entre $\text{MIN_WORKERS}=1$ y $\text{MAX_WORKERS}=16$. La prueba por ráfagas muestra tres picos diferenciados: 30.000 mensajes llevan el sistema a 16 consumidores, 8.000 mensajes provocan un pico de 13 consumidores y 3.000 mensajes provocan un pico de 5 consumidores.

Después de cada ráfaga, cuando la cola queda vacía, el sistema no mata todos los workers de golpe. Reduce progresivamente el número de consumidores hasta volver al mínimo. Esto es positivo porque evita oscilaciones bruscas y demuestra un comportamiento estable de escalado/desescalado.

7. Discusión comparativa

La arquitectura directa tiene la ventaja de ser simple y ofrecer una respuesta inmediata al cliente. Es adecuada cuando se necesita confirmación síncrona de cada operación. Sin embargo, sus resultados muestran que el throughput queda limitado rápidamente por componentes comunes del camino HTTP.

La arquitectura indirecta es más compleja, porque añade RabbitMQ y requiere separar publicación y procesamiento, pero ofrece mucho mejor rendimiento bajo carga. El sistema puede absorber ráfagas grandes de mensajes, escalar consumidores y procesar de forma asíncrona. Esto la hace más adecuada para escenarios de alta concurrencia.

En términos de consistencia, ambas arquitecturas se comportan correctamente gracias a Redis. Para unnumbered se usa un inventario compartido que impide vender más entradas de las disponibles. Para numbered se usa una operación atómica tipo SETNX, de modo que solo la primera solicitud que consigue marcar un asiento tiene éxito.

En términos de contención, la indirecta mantiene un throughput mucho más alto. En parte esto se debe a que RabbitMQ reparte trabajo entre consumers y a que los fallos por asientos ya vendidos se resuelven rápido. Si se introdujera una sección crítica más larga o locks pesimistas, el impacto de hotspot sería más visible.

8. Limitaciones y posibles mejoras

Los resultados están condicionados por el entorno de red usado: VM AWS Academy como cliente, torre como servidor y conexión por Tailscale. Por tanto, los valores absolutos pueden variar si se ejecuta en otra red o en máquinas distintas.

En la prueba de escalado indirecto se usaron 50.000 solicitudes unnumbered con un inventario de 20.000 entradas. Esto generó 30.000 rechazos esperados. El benchmark original marcó esas filas como FAIL por una expectativa incorrecta, pero los datos son válidos para medir throughput y consistencia.

Como mejora futura, se podrían añadir métricas de latencia end-to-end en la arquitectura indirecta, incluyendo el tiempo desde publicación del mensaje hasta procesamiento final, no solo throughput global.

9. Conclusiones

Las pruebas confirman que las dos arquitecturas implementadas son correctas: no se produce overselling ni ventas duplicadas. La arquitectura directa es funcional y sencilla, pero su escalabilidad queda limitada en el entorno evaluado. La arquitectura indirecta con RabbitMQ escala mucho mejor y alcanza miles de operaciones por segundo al aumentar los workers.

El autoscaler directo demuestra adaptación a carga, aunque con cierto sobreescalado. El autoscaler indirecto muestra un comportamiento más claro: escala según el backlog de RabbitMQ, alcanza el máximo con cargas grandes y vuelve progresivamente al mínimo durante las pausas.

En conjunto, la solución indirecta es la opción más adecuada para una plataforma de venta de entradas con ráfagas intensas de usuarios, mientras que la directa puede ser suficiente para cargas más moderadas o cuando se prioriza simplicidad.

Anexo A. Ficheros de resultados utilizados

Resultados directos: resultados_super_stress_direct y resultados_direct_autoscaler.

Resultados indirectos: resultados_indirect_no_autoscaler y resultados_indirect_autoscaler.

Los CSV se han usado para generar las tablas y gráficas incluidas en este informe.